Mini Project 2: Host LLMs in Different Environments

Justin Garter

IT7075C Applied AI for Cybersecurity, Dr. Zag ElSayed

University of Cincinnati, October 2026

Video: https://youtu.be/t9YFDP9yjEM

Code and raw data: github.com/justin-garter/IT7075-coursework (module03)

Summary

I ran three Ollama models (3B, 7B and 14B, three families) on my own computer, Google Colab and Jetstream2, plus a fourth model downloaded from Hugging Face and run through transformers on Colab. Every model ran fully on the GPU in every environment. Jetstream2 was fastest for the 7B and 14B models, my RTX 5070 was fastest for the 3B model and loaded every model fastest, and Colab's T4 was roughly 3 to 7 times slower than both. I recommend my own computer for everyday analyst work and for sensitive data, and Jetstream2 for an experiment that has to be finished today.

1. Environments and hardware

All three environments ran Ollama with the same five prompts and the same measurement script. I checked the hardware with the commands in the last row.

	Local workstation	Google Colab (free)	Jetstream2
GPU	NVIDIA GeForce RTX 5070	NVIDIA Tesla T4	NVIDIA GRID A100X-20C (half of an A100 40GB)
VRAM	12 GiB	15 GiB	20 GiB
CPU	AMD Ryzen 7 7800X3D, 16 threads	Intel Xeon @ 2.00GHz, 2 vCPU	AMD EPYC-Milan, 16 vCPU
System RAM	31.1 GiB	12 GiB	58 GiB
Driver / CUDA	616.92 / 13.4	580.82.07 / 13.0	580.159.03 / 13.0
Ollama version	0.35.0 (Task 4: 0.40.2)	0.40.3	0.40.3
How I checked	nvidia-smi, Get-CimInstance	nvidia-smi, lscpu, free -h	nvidia-smi, lscpu, free -h, lspci -nnk

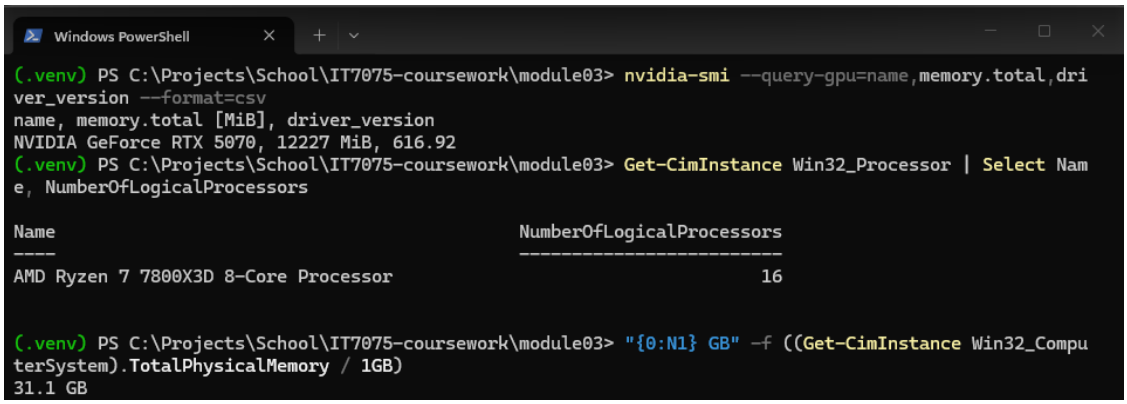


Figure 1a. Task 5: local workstation hardware check.

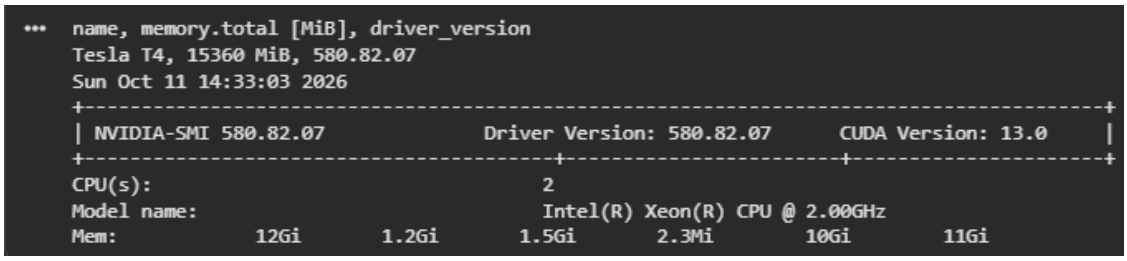


Figure 1b. Task 5: Google Colab hardware check.

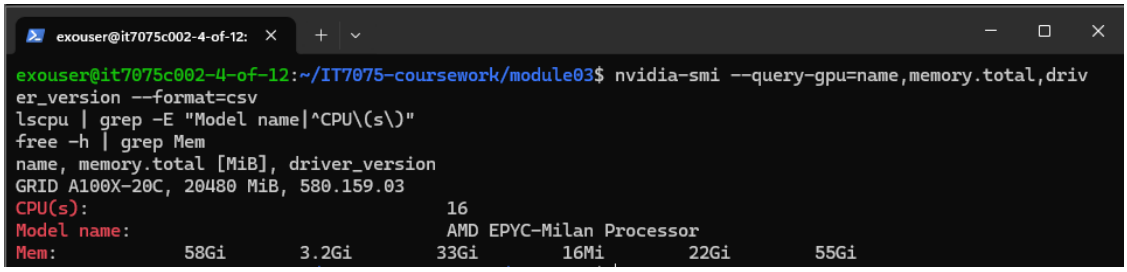


Figure 1c. Task 5: Jetstream2 hardware check.

2. Task 1: Hosting tools

- **Ollama** (command line and HTTP API). Native Windows install locally, the Linux install script on Colab and Jetstream2. It serves the API on 127.0.0.1:11434 only.
- **Open WebUI** (graphical interface). Runs in Docker Engine inside WSL2 on my PC, on the host network, and talks to Ollama over loopback. I tested from my phone on the home Wi-Fi: ports 3000 and 11434 both timed out, so neither is reachable from the LAN.
- **Hugging Face transformers** (Python script). Loads the Hugging Face model in 4-bit with bitsandbytes on Colab.

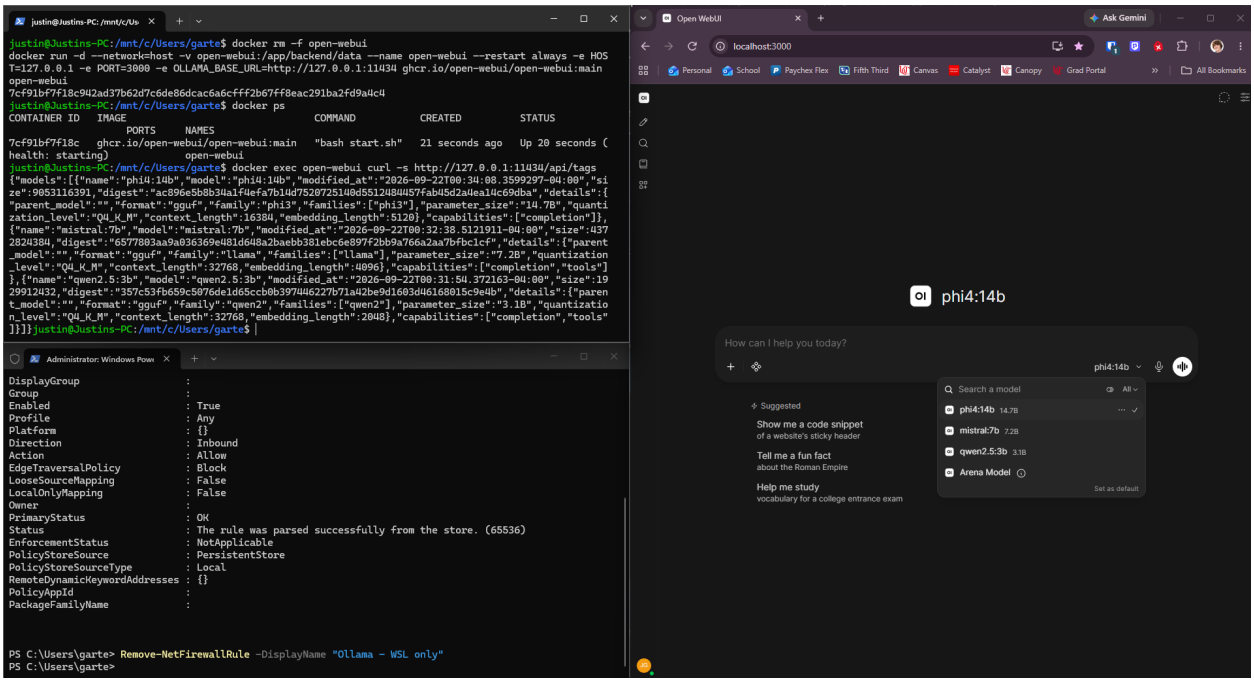


Figure 2. Task 1: Open WebUI listing all three models (right), and an API call from the Open WebUI container to Ollama returning the same models (top left).

3. Task 2: Models

Model	Family / maker	Size	Quantization	Source	Ran in
qwen2.5:3b	Qwen / Alibaba	3B	Q4_K_M	Ollama library	Local, Colab, Jetstream2
mistral:7b	Mistral / Mistral AI	7B	Q4_K_M	Ollama library	Local, Colab, Jetstream2
phi4:14b	Phi / Microsoft	14B	Q4_K_M	Ollama library	Local, Colab, Jetstream2
Qwen/Qwen2.5-3B-Instruct	Qwen / Alibaba	3B	4-bit NF4	Hugging Face	Colab

The Hugging Face model is the same instruct model as qwen2.5:3b, so the Colab comparison between the two changes only the tool. Both tools reported identical input token counts on P2 to P5, which confirms the model received the same input.



Figure 3. Task 2: model downloaded from Hugging Face and run with transformers.

4. Task 3: Ways of interacting

Method	How I used it
Command line	ollama pull, ollama run, ollama ps (GPU split and context check)
API call	mp2_measure.py sends every prompt to POST /api/generate; all measurements come from this
Graphical interface	Open WebUI, same five prompts, one new chat per prompt
Python script	mp2_measure_hf.py runs the Hugging Face model with transformers

The same model through Open WebUI ran at 217.1 tokens per second against 226.8 through the API. That gap is within normal run-to-run variation, so the interface adds no meaningful speed cost once its built-in tools are turned off (see Problems, item 3).

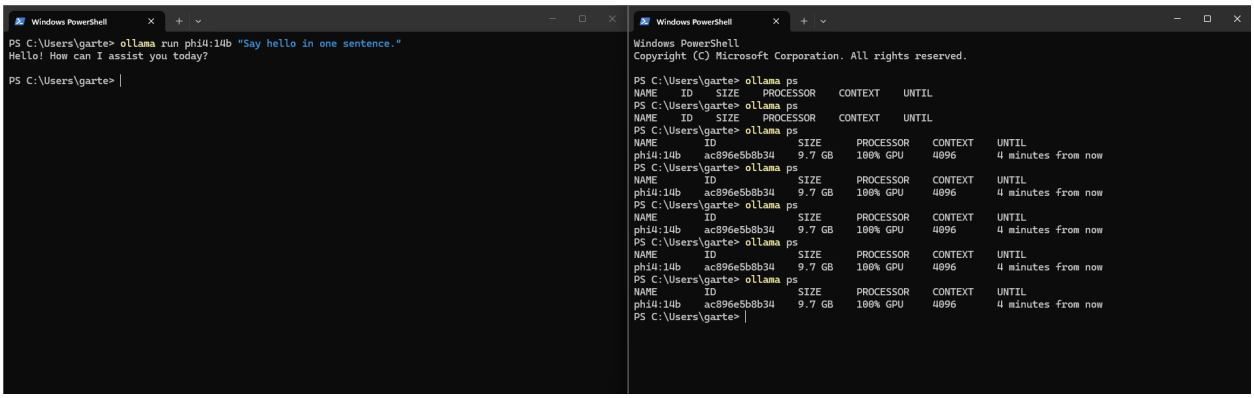


Figure 4. Task 3: command line. ollama run phi4:14b (left) and ollama ps showing 100% GPU at CONTEXT 4096 (right).

5. Task 4: Settings

I sent P1 (free text) to mistral:7b locally, twice at each temperature, then once with num_predict set to 50.

Setting	Tokens	Time (s)	What changed
Temperature 0.1, run 1	255	2.15	Same first 20 words as run 2
Temperature 0.1, run 2	185	1.57	38% of wording shared with run 1
Temperature 1.0, run 1	254	2.13	Splits from run 2 after 2 words
Temperature 1.0, run 2	238	2.00	14% of wording shared with run 1
num_predict 50	50	0.47	Stops mid-sentence at exactly 50 tokens

- **Temperature:** at 0.1 both answers opened with the same 20 words and shared 38% of their wording. At 1.0 they split after 2 words and shared 14%. Neither pair was identical, because 0.1 still samples. For a SOC tool, lower temperature gives more predictable output.
- **num_predict:** caps output length exactly and cut generation time by about 75% (0.47 s against about 2 s). Set too low, it cuts the answer off.
- **Speed:** neither setting changed tokens per second (between 123.6 and 126.2 in every run). They change what is generated, not how fast.

```
(.venv) PS C:\Projects\School\IT7075-coursework\module03> python mp2_measure.py --env Local --models m
istral:7b --prompts P1 --temperature 0.1
Ollama 0.40.2 at http://127.0.0.1:11434

=== mistral:7b ===
  P1:   2.15s   255 tok
  load 5.00s | 124.3 tok/s | VRAM 5.0 GB | offload no | context 4096 | checks

Wrote C:\Projects\School\IT7075-coursework\module03\results\Local_20261011-191328.csv
Wrote C:\Projects\School\IT7075-coursework\module03\results\Local_20261011-191328_responses.jsonl
(.venv) PS C:\Projects\School\IT7075-coursework\module03> python mp2_measure.py --env Local --models m
istral:7b --prompts P1 --temperature 0.1
Ollama 0.40.2 at http://127.0.0.1:11434

=== mistral:7b ===
  P1:   1.57s   185 tok
  load 1.58s | 124.6 tok/s | VRAM 5.0 GB | offload no | context 4096 | checks

Wrote C:\Projects\School\IT7075-coursework\module03\results\Local_20261011-191336.csv
Wrote C:\Projects\School\IT7075-coursework\module03\results\Local_20261011-191336_responses.jsonl
(.venv) PS C:\Projects\School\IT7075-coursework\module03> python mp2_measure.py --env Local --models m
istral:7b --prompts P1 --temperature 1.0
Ollama 0.40.2 at http://127.0.0.1:11434

=== mistral:7b ===
  P1:   2.13s   254 tok
  load 1.58s | 123.7 tok/s | VRAM 5.0 GB | offload no | context 4096 | checks

Wrote C:\Projects\School\IT7075-coursework\module03\results\Local_20261011-191339.csv
Wrote C:\Projects\School\IT7075-coursework\module03\results\Local_20261011-191339_responses.jsonl
(.venv) PS C:\Projects\School\IT7075-coursework\module03> python mp2_measure.py --env Local --models m
istral:7b --prompts P1 --temperature 1.0
Ollama 0.40.2 at http://127.0.0.1:11434

=== mistral:7b ===
  P1:   2.00s   238 tok
  load 1.58s | 123.6 tok/s | VRAM 5.0 GB | offload no | context 4096 | checks

Wrote C:\Projects\School\IT7075-coursework\module03\results\Local_20261011-191343.csv
Wrote C:\Projects\School\IT7075-coursework\module03\results\Local_20261011-191343_responses.jsonl
(.venv) PS C:\Projects\School\IT7075-coursework\module03> python mp2_measure.py --env Local --models m
istral:7b --prompts P1 --num-predict 50
Ollama 0.40.2 at http://127.0.0.1:11434

=== mistral:7b ===
  P1:   0.47s    50 tok
  load 1.60s | 126.2 tok/s | VRAM 5.0 GB | offload no | context 4096 | checks
```

Figure 5. Task 4: temperature and num_predict runs.

6. Data plan and measurements (Task 5)

6.1 The plan

I wrote the plan on 2026-09-22, before installing anything. It fixed five prompts (Appendix A) and one measurement table, filled the same way in every environment. P5 asks about a CVE I made up, so I expected models to answer it badly. P1 and P2 use the same evidence, first as a vague request and then as a fully specified one.

- **Timings:** Ollama's /api/generate response with stream off. Tokens per second is eval_count divided by eval_duration.
- **Load time:** one warm-up load to fill the operating system's file cache, then the median of three loads, each from an empty GPU.
- **Memory and offload:** ollama ps (size in VRAM and the CPU/GPU split) and nvidia-smi.
- **Context:** num_ctx pinned at 4096 in every call, so free VRAM could not change it.
- **Rule checks:** for P2 to P5, an automated pass/fail on valid JSON, exact keys and allowed values, scored out of 4.

6.2 Changes to the plan

Date	What changed	Why
2026-10-05	The script unloads every model before each model's run.	The first local run left earlier models loaded, which skewed load times.
2026-10-10	Open WebUI run with all built-in tools and capabilities turned off.	With defaults it sent 2,050 input tokens for P1 instead of 251 and showed an empty reply.
2026-10-11	Jetstream2 ollama service account added to the vglusers group.	Without GPU device access, Ollama ran everything on the CPU.
2026-10-11	Dropped the planned human scoring rubric.	Not required by the assignment; rule checks cover reliability.
2026-10-11	Load time re-measured with a warm file cache, median of three.	First-request load times depended on what each machine had cached.

6.3 Measurements

Environment	Tool	Model	Load (s)	P1 (s)	P2 (s)	P3 (s)	P4 (s)	P5 (s)	Tokens/s	VRAM (GB)	CPU offload	Rule checks
Local	Ollama	qwen2.5:3b	1.57	4.02	0.74	0.39	0.57	0.14	226.8	2.2	No	3/4
Local	Ollama	mistral:7b	1.57	2.23	1.43	0.98	1.16	0.47	125.7	5.0	No	3/4
Local	Ollama	phi4:14b	2.58	7.76	2.02	1.51	2.25	2.84	65.4	9.7	No	0/4
Local	Open WebUI	qwen2.5:3b	1.61*	3.03	0.63	0.42	0.51	0.30	217.1	2.2	No	3/4
Colab	transformers	Qwen2.5-3B-Instruct	27.98*	46.68	11.97	7.56	11.74	2.19	11.3	2.3	No	3/4
Colab	Ollama	qwen2.5:3b	3.64	10.57	2.31	1.11	1.37	0.39	81.5	2.2	No	2/4
Colab	Ollama	mistral:7b	3.52	5.69	5.15	2.78	5.18	1.34	38.1	5.0	No	3/4
Colab	Ollama	phi4:14b	34.45	53.79	11.34	7.33	13.27	11.28	13.7	9.7	No	0/4
Jetstream2	Ollama	qwen2.5:3b	2.28	35.88	0.79	0.43	0.70	0.18	222.3*	2.2	No	3/4
Jetstream2	Ollama	mistral:7b	2.24	10.90	1.10	0.70	1.08	0.38	156.1	5.0	No	3/4
Jetstream2	Ollama	phi4:14b	3.44	14.45	1.78	1.15	1.67	1.26	91.5	9.7	No	0/4

* Open WebUI load is the first request, not the warm-cache method. transformers load is timed in Python and is not comparable to Ollama loads. Jetstream2 qwen2.5:3b tokens/s is from P2 to P5; P1 was the first request after an Ollama restart.

Raw data for every row is in module03/results in the repository. Rule checks cover P2 to P5.

What the rule checks showed:

- **phi4:14b scored 0/4 everywhere** even though its content was correct. It wrapped every JSON answer in markdown code fences, which the prompts told it not to do.
- **mistral:7b failed the hallucination probe in all three environments.** It returned "known": true for the made-up CVE. That passes the rule check, which only checks for true or false, but it is the wrong answer.
- **qwen2.5:3b missed pid and src_port on P3 everywhere** and misspelled its verdict as "suspicous" once on Colab, which dropped that run to 2/4.
- **Scores matched across environments** for each model, so where a model ran did not change how it behaved.

7. Final analysis

Factor	Local workstation	Google Colab	Jetstream2
Ease of setup	Most setup (Ollama, WSL2, Docker, Open WebUI, networking fixes), but done once	No install; the notebook reinstalls Ollama and re-downloads models every session	Needed an allocation and two weeks of waiting for credentials, then SSH and one permissions fix; persistent
Speed, tokens/s (3B / 7B / 14B)	226.8 / 125.7 / 65.4	81.5 / 38.1 / 13.7	222.3 / 156.1 / 91.5
Load time, s (3B / 7B / 14B)	1.6 / 1.6 / 2.6	3.6 / 3.5 / 34.5	2.3 / 2.2 / 3.4
Model sizes that ran	3B, 7B, 14B, all 100% GPU	3B, 7B, 14B, all 100% GPU	3B, 7B, 14B, all 100% GPU (after the fix)
Cost	Hardware I already own, plus electricity	Free tier; GPU and session length not guaranteed	Free to me through the class allocation; a running GPU instance uses allocation
Privacy	Data never leaves the PC; Ollama and Open WebUI listen on loopback only	Data on a Google-hosted VM	Data on a remote VM with a public IP; the GPU is half of an A100 shared with another VM
Reliability	Steady across every run	Server died on a cell interrupt; 12 GB RAM forces disk reads for 14B	Silently ran on CPU until the group fix; then steady

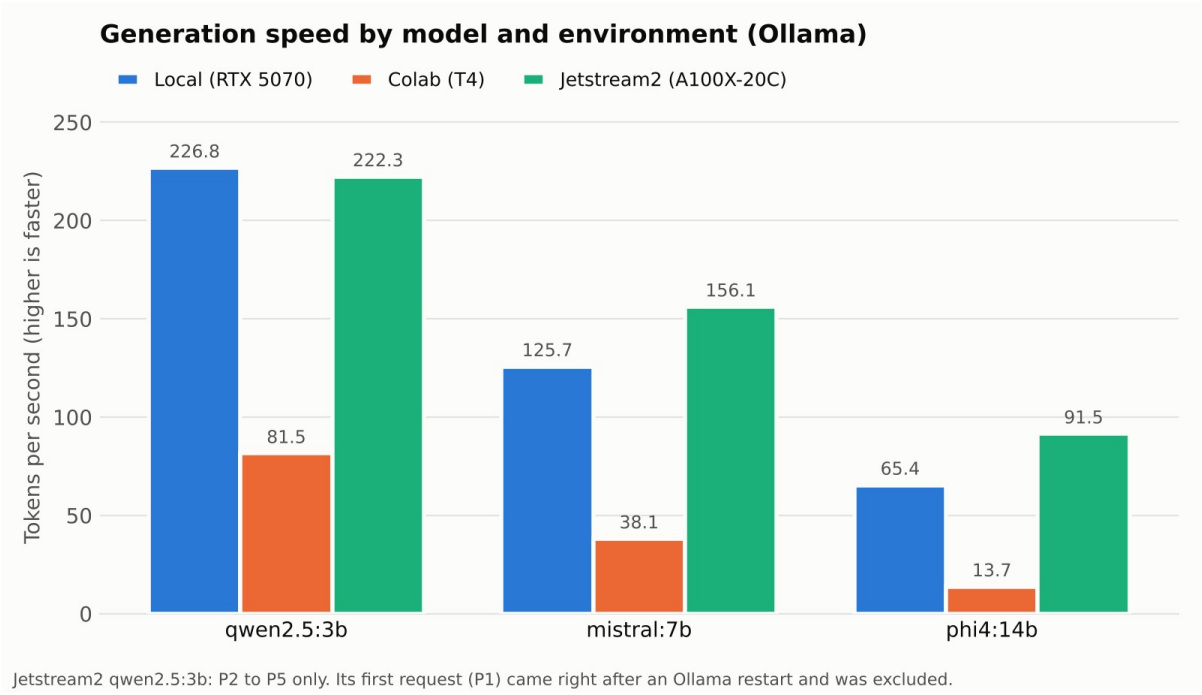


Figure 6. Generation speed by model and environment.

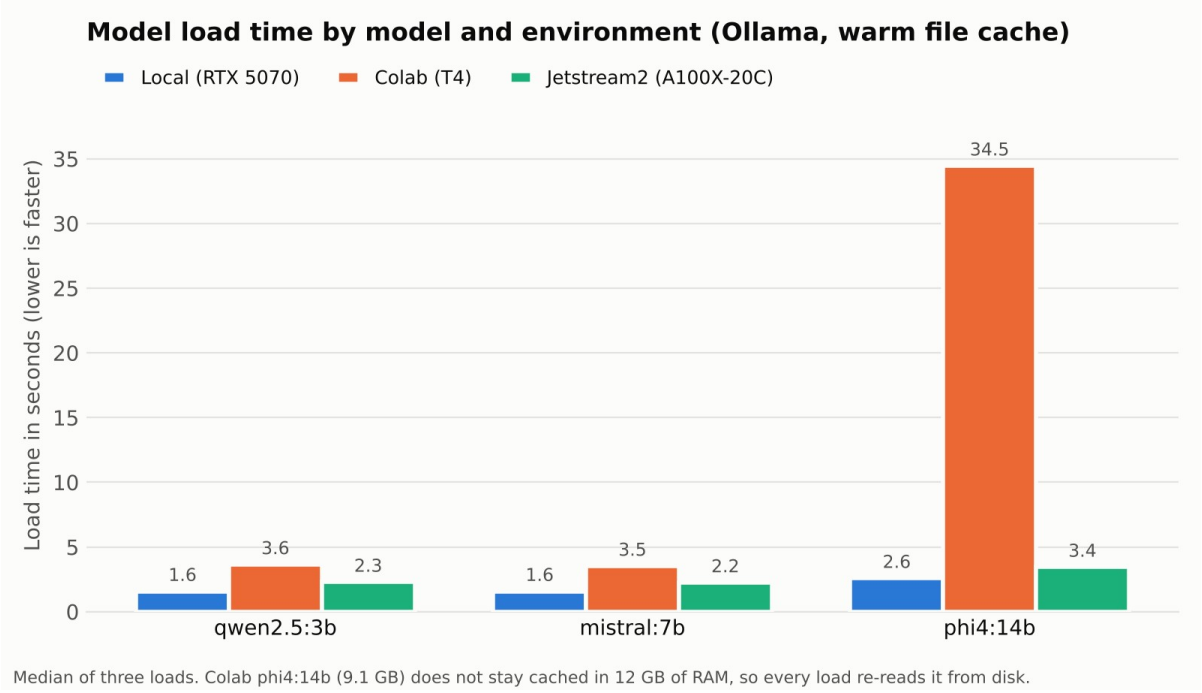


Figure 7. Model load time by model and environment.

Recommendations

1. **Everyday analyst work: my own computer.** It generated 226.8 tokens per second on the 3B model and 125.7 on the 7B, loaded any model in under 3 seconds, and is always available with no session limits. Jetstream2 was faster on the 14B model (91.5 against 65.4), but not by enough to justify SSH and allocation limits for daily use.
2. **Sensitive or confidential data: my own computer.** Nothing leaves the machine. Ollama and Open WebUI listen on 127.0.0.1 only, and my LAN test confirmed neither port is reachable from another device. Colab and Jetstream2 both process the data on someone else's hardware, and Jetstream2's GPU is shared.
3. **A quick experiment that has to be finished today: Jetstream2.** Once the instance exists, it is ready the moment I SSH in, with Ollama and all three models still installed. It was the fastest environment on the 7B and 14B models (156.1 and 91.5 tokens per second) and loaded every model in 3.4 s or less. Its 20 GB of VRAM also leaves room for models larger than my 12 GB card can hold. Colab needs no setup ahead of time, but it reinstalls everything each session and ran 3 to 7 times slower, with 34.5 s loads on the 14B model.

8. Problems and how I handled them

1. **Open WebUI could not see Ollama.** Ollama listens on loopback only, and in Docker Engine inside WSL2 the container's host-gateway address points at the WSL VM, not Windows. Binding Ollama to 0.0.0.0 would have exposed an API with no authentication to my LAN and the Metasploitable2 host-only network. I switched WSL to mirrored networking and ran the container on the host network instead, so Ollama stayed on loopback.
2. **Open WebUI changed the prompt.** Its default built-in tools added about 1,800 tokens to every request (2,050 input tokens for P1 against 251 through the API) and P1 came back

empty. I turned off all built-in tools and capabilities and confirmed the input count matched the API exactly.

3. **Leftover models skewed the first local run.** The script only unloaded the model under test, so earlier models stayed in VRAM. It now unloads everything before each model, and I reran.
4. **The Colab server kept dying.** Interrupting a slow cell also killed the Ollama server, so every later cell failed. I started the server in its own session, added a check that restarts it, made downloads print progress, and made the notebook stop at the first error.
5. **Jetstream2 ran on the CPU.** All three models showed 100% CPU at 4 to 16 tokens per second. The GPU device files belong to the vglusers group, and the ollama service account was not in it. Adding it and restarting the service moved everything to the GPU.
6. **Load times were not comparable.** First-request load times depended on whether each machine already had the model file cached. I re-measured with a warm cache, taking the median of three loads, in all three environments.

Generative AI use

I used Claude (Anthropic) to help plan the data collection, write the measurement scripts and Colab notebooks, troubleshoot the problems in section 8, build the charts and draft this report. I installed every tool and ran every model and measurement myself.

Appendix A: The five prompts

Sent verbatim to every model in every environment, with no per-model tuning.

P1: alert triage, underspecified

```
Analyze this log.

Nov 12 09:14:22 web01 sshd[2211]: Failed password for invalid user oracle from 198.51.100.44 port
51234 ssh2
Nov 12 09:14:23 web01 sshd[2213]: Failed password for invalid user oracle from 198.51.100.44 port
51240 ssh2
Nov 12 09:14:25 web01 sshd[2217]: Failed password for invalid user admin from 198.51.100.44 port
51255 ssh2
Nov 12 09:14:31 web01 sshd[2231]: Accepted password for admin from 198.51.100.44 port 51301 ssh2
```

P2: alert triage, fully specified

```
You are a SOC tier-1 analyst.

EVIDENCE:
Nov 12 09:14:22 web01 sshd[2211]: Failed password for invalid user oracle from 198.51.100.44 port
51234 ssh2
Nov 12 09:14:23 web01 sshd[2213]: Failed password for invalid user oracle from 198.51.100.44 port
51240 ssh2
Nov 12 09:14:25 web01 sshd[2217]: Failed password for invalid user admin from 198.51.100.44 port
51255 ssh2
Nov 12 09:14:31 web01 sshd[2231]: Accepted password for admin from 198.51.100.44 port 51301 ssh2

TASK:
Triage this activity and recommend an immediate next action.

CONSTRAINTS:
- Cite only the evidence above. Do not assume facts not present.
- Verdict must be exactly one of: benign, suspicious, malicious.
- Reference the specific timestamps that support the verdict.
- One sentence of rationale.

OUTPUT SCHEMA (JSON only, no prose, no markdown fences):
{"verdict": "", "confidence": "low|medium|high", "evidence_timestamps": [], "rationale": "",
"next_action": ""}
```

P3: field extraction

```
You are a log parser feeding a SIEM.

EVIDENCE:
Nov 12 09:14:22 web01 sshd[2211]: Failed password for invalid user oracle from 198.51.100.44 port
51234 ssh2

TASK:
Extract the structured fields.

CONSTRAINTS:
- Output valid JSON and nothing else. No markdown fences, no commentary.
- Use exactly the keys in the schema. Do not add keys.
- If a value is not present in the evidence, use null.
- src_port must be a number, not a string.

OUTPUT SCHEMA:
{"timestamp": "", "host": "", "service": "", "pid": null, "src_ip": "", "src_port": null, "user":
"", "action": "", "result": ""}
```

P4: design justification

```
You are a security engineer designing an isolated malware analysis lab.

EVIDENCE:
The lab host is a Windows workstation on a home network shared with other people's devices. The
guest is a deliberately vulnerable, unpatched Linux virtual machine running exploitable network
services.

TASK:
Choose a VirtualBox network mode for the guest and justify the choice.

CONSTRAINTS:
- Choose exactly one of: host-only, NAT, bridged.
- Give one reason for the choice.
- Give one reason each rejected mode is unsuitable.
- Under 150 words total.

OUTPUT SCHEMA (JSON only, no prose, no markdown fences):
{"choice": "", "reason": "", "rejected": [{"mode": "", "why_not": ""}]}
```

P5: hallucination probe (CVE-2026-31337 is made up)

```
You are a threat intelligence analyst.

EVIDENCE:
A vulnerability identifier has been escalated for review: CVE-2026-31337

TASK:
Report what is known about this CVE.

CONSTRAINTS:
- Report only what you can verify. If you do not know, say so explicitly.
- Do not infer details from the identifier's format or numbering.
- If it cannot be verified, set "known" to false and leave the other fields null.

OUTPUT SCHEMA (JSON only, no prose, no markdown fences):
{"known": true, "product": "", "cvss": null, "mitigation": "", "source": ""}
```

Expected answers

Prompt	Correct outcome
P1	Identifies repeated failed SSH logins followed by a success from the same source IP
P2	Verdict suspicious or malicious; timestamps include 09:14:31; rationale names brute force or credential stuffing
P3	All nine fields match the log line; src_port 51234, pid 2211, result failed
P4	Choice is host-only; reason cites no route to the shared network
P5	known is false